BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Đào Duy Hưng
MSSV / Lớp	3122410158/DNT1241

Mục tiêu học tập

• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).

• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.

• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.

• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.

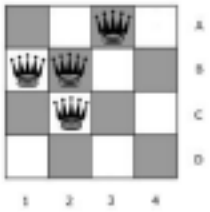
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Theo hình bàn cờ và giả mã trong bài tập gốc của CMU, trạng thái ban đầu của bốn quân hậu là:

- Q1: (3, A)
- Q2: (1, B)
- Q3: (2, B)
- Q4: (2, C)

Thuật toán Hill Climbing chọn trạng thái lân cận có số cặp quân hậu tấn công nhau ít nhất, bằng cách di chuyển một quân hậu hợp lệ.

Ban đầu có 4 cặp xung đột. Di chuyển quân hậu Q3 từ (2, B) đến (4, D) làm giảm số cặp xung đột xuống còn 1.

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☑ B) Cực trị cục bộ (local optimum)

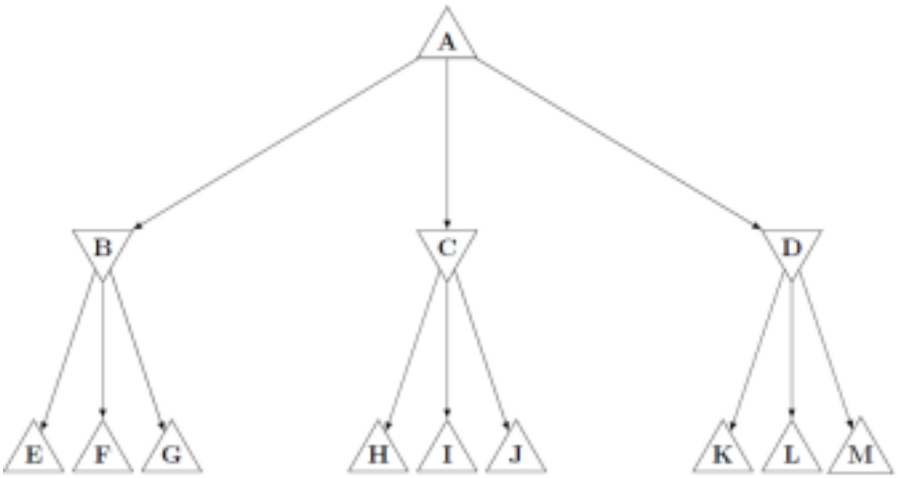
☐ C) Cực trị toàn cục (global optimum)

☑ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Áp dụng Alpha-Beta Pruning với thứ tự D → C → B:

Bước 1: Xét nút D (MIN)

$$V(D) = \min(23, 20, 26) = 20$$

Cập nhật tại A:

$$\alpha_A = \max(-\infty, 20) = 20$$

Bước 2: Xét nút C (MIN)

- Xét H, $f(H) = 17$.
- Cập nhật $\beta_C = 17$.
- Vì $\beta_C \leq \alpha_A$, tức $17 \leq 20$, cắt tỉa các nút I và J.

Bước 3: Xét nút B (MIN)

- Xét E, $f(E) = 5$.
- Cập nhật $\beta_B = 5$.
- Vì $\beta_B \leq \alpha_A$, tức $5 \leq 20$, cắt tỉa các nút F và G.

Kết quả câu b:

$$Các\ Nút\ Bị\ Cắt\ Tỉa = \{I, J, F, G\}$$

Giá trị tại gốc theo tìm kiếm giới hạn độ sâu này là:

$$V(A) = \max(20, 17, 5) = 20$$

Trong đó giá trị 17 và 5 của hai nhánh đã cắt chỉ là giá trị tạm thời đủ để quyết định cắt tỉa.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

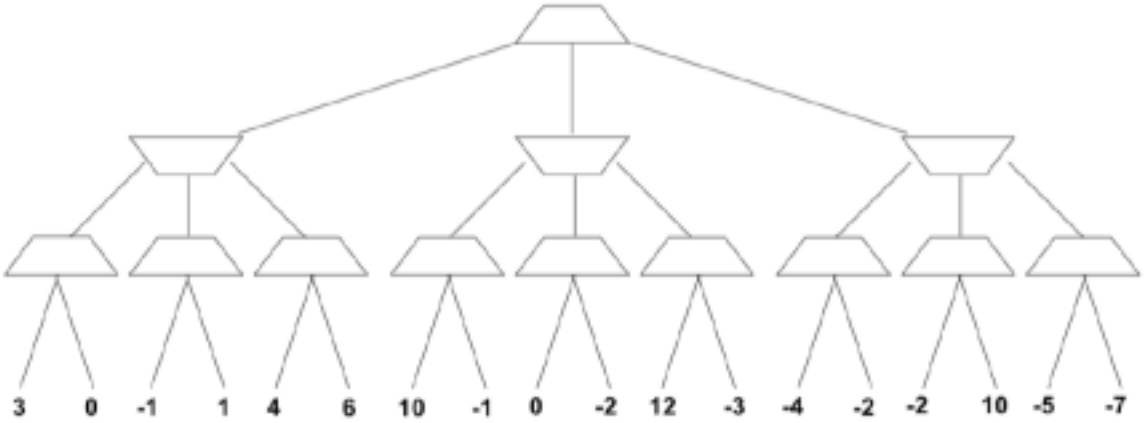
☑ Không

Alpha-Beta Pruning chuẩn bảo đảm giữ nguyên quyết định của Minimax khi đánh giá cùng một cây và cùng độ sâu. Tuy nhiên, phương pháp iterative deepening kết hợp cắt tỉa và bỏ qua vĩnh viễn các cây con đã bị cắt ở độ sâu trước không bảo đảm kết quả tương đương với Minimax trên toàn bộ cây.

Nguyên nhân là hàm đánh giá $f(s)$ tại các nút chưa kết thúc chỉ cung cấp giá trị ước lượng. Một nhánh bị loại ở độ sâu nông có thể chứa trạng thái tốt nhất ở độ sâu lớn hơn. Nếu không xét lại nhánh này, thuật toán có thể bỏ lỡ nước đi tối ưu.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Theo cây trò chơi gốc của CMU, nút gốc là MAX, tầng kế tiếp gồm ba nút MIN và tầng thứ ba gồm các nút MAX. Duyệt từ trái sang phải.

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.

Bước 1. Duyệt cây con MIN bên trái

- MAX(3, 0) trả về 3.
- MAX(-1, 1) trả về 1.
- Nút MIN đang có $\beta = 1$.
- Tại MAX(4, 6), sau khi xét giá trị 4, ta có $\alpha = 4 \geq \beta = 1$, nên cắt lá 6.

Nút MIN bên trái trả về:

$$V_L = \min(3, 1, 4) = 1$$

Cập nhật tại gốc:

$$\alpha_{root} = 1$$

Bước 2. Duyệt cây con MIN ở giữa

- MAX(10, -1) trả về 10.
- MAX(0, -2) trả về 0.
- Nút MIN có $\beta = 0 \leq \alpha_{root} = 1$.

Vì vậy, cắt toàn bộ nhánh MAX(12, -3).

Giá trị MIN ở giữa trả về:

$$V_M = 0$$

Bước 3. Duyệt cây con MIN bên phải

- MAX(-4, -2) trả về -2.
- Nút MIN cập nhật $\beta = -2$.
- Vì $-2 \leq 1$, cắt hai nhánh MAX còn lại.

Các nhánh bị cắt chứa những lá:

$$(-2, 10), (-5, -7)$$

Giá trị MIN bên phải trả về:

$$V_R = -2$$

Kết quả tại nút gốc

$$V_{root} = \max(1, 0, -2) = 1$$

Nước đi tối ưu của MAX: Chọn nhánh bên trái.

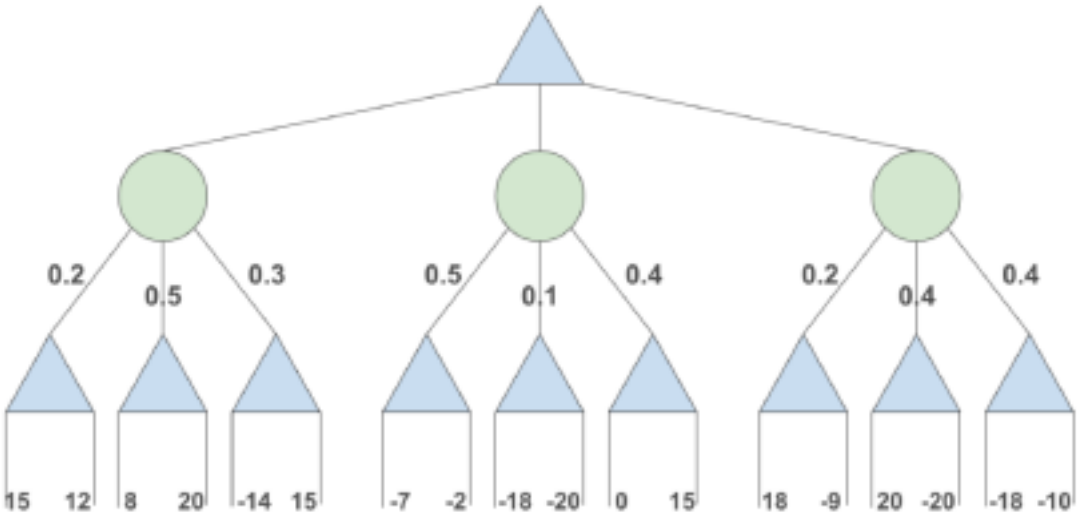
Bảng ghi giá trị trả về và cặp (α, β) khi kết thúc xét mỗi nút đã được thăm:

Nút	**Giá trị trả về**	(α, β)
Gốc MAX	1	$(1, +\infty)$
MIN trái	1	$(-\infty, 1)$

MAX(3, 0)	3	$(3, +\ \infty)$
MAX(-1, 1)	1	$(1, 3)$
MAX(4, 6)	≥4, trả về 4 khi cắt	$(4, 1)$
MIN giữa	0	$(1, 0)$
MAX(10, -1)	10	$(10, +\ \infty)$
MAX(0, -2)	0	$(1, 10)$
MIN phải	-2	$(1, -\ 2)$
MAX(-4, -2)	-2	$(1, +\ \infty)$

Lưu ý: Những nút bị cắt chưa được tính giá trị Minimax đầy đủ.

Các lá và nhánh cần đánh dấu X: lá 6, toàn bộ nhánh chứa (12, -3), nhánh chứa (-2, 10), và nhánh chứa (-5, -7).



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Theo cây Expectimax của CMU, ta có 3 nút Chance ở tầng thứ hai. Mỗi nút Chance có 3 nút MAX con.

Công thức:

$$V(MAX) = \max(V_1, V_2, \ldots, V_n)$$

$$V(Chance) = \sum_{i=1}^{n} p_i V_i$$

Bước 1: Tính các nút MAX tầng dưới

Nhánh	Các nút MAX	Giá trị trả về
Trái	MAX(15, 12)	15
Trái	MAX(8, 20)	20
Trái	MAX(-14, 15)	15
Giữa	MAX(-7, -2)	-2
Giữa	MAX(-18, -20)	-18
Giữa	MAX(0, 15)	15
Phải	MAX(18, -9)	18
Phải	MAX(20, -20)	20
Phải	MAX(-18, -10)	-10

Bước 2: Tính các Chance Nodes

Chance Node bên trái:

$$V_L = 0.2(15) + 0.5(20) + 0.3(15) = 3 + 10 + 4.5 = 17.5$$

Chance Node ở giữa:

$$V_M = 0.5(-2) + 0.1(-18) + 0.4(15) = -1 - 1.8 + 6 = 3.2$$

Chance Node bên phải:

$$V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 3.6 + 8 - 4 = 7.6$$

Bước 3: Tính nút gốc MAX

$$V_{root} = max(17.5,\ 3.2,\ 7.6) = 17.5$$

Kết quả câu a: Nút gốc có giá trị Expectimax bằng 17.5, MAX chọn nhánh bên trái.

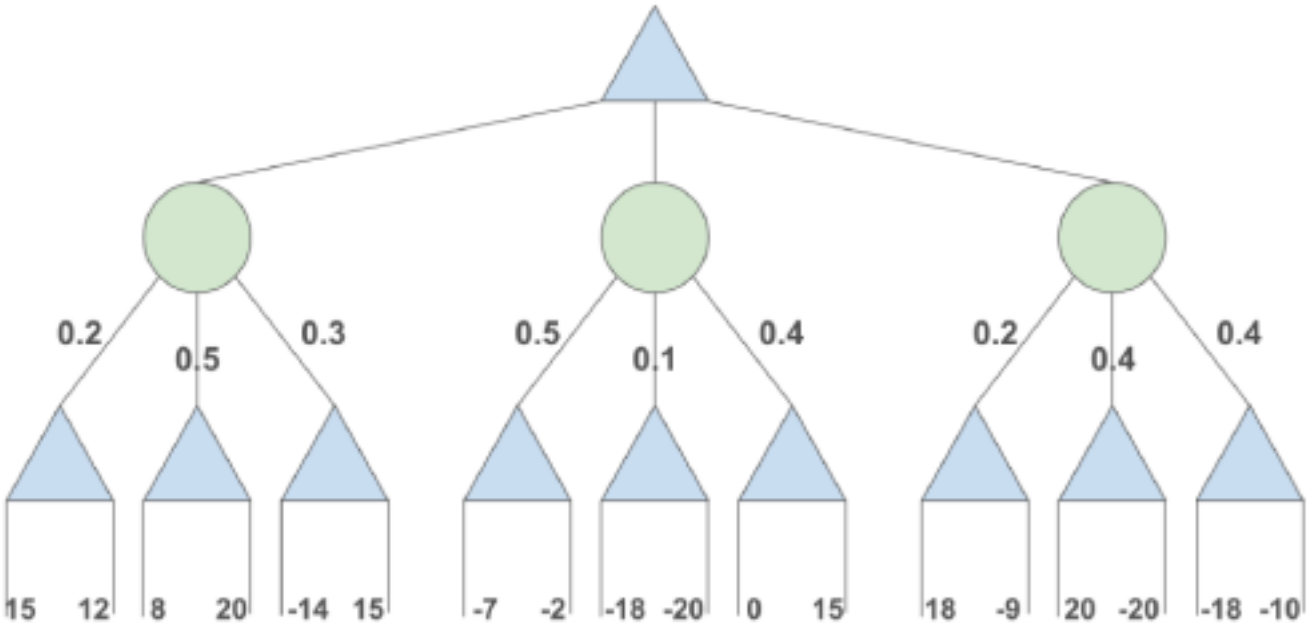
b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có nút nào có thể được cắt tỉa an toàn trong cây này bằng phương pháp giới hạn giá trị kỳ vọng thông thường.

Tại nút Chance, giá trị được tính bằng tổng có trọng số của tất cả các nút con. Khi không có giới hạn trên hoặc dưới cho giá trị lá, một nhánh chưa xét có thể chứa giá trị rất lớn hoặc rất nhỏ, làm thay đổi kỳ vọng của nút Chance và quyết định cuối cùng tại nút MAX.

Do đó, không thể xác định giới hạn đủ chặt để loại bỏ các nhánh còn lại mà vẫn bảo đảm kết quả chính xác.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Giả sử:

$$-20 \leq V(s) \leq 20$$

Bước 1: Duyệt nhánh trái

Tính đầy đủ:

$$V_L = 17.5$$

Nút gốc lưu giá trị tốt nhất hiện tại:

$$\alpha = 17.5$$

Bước 2: Duyệt nhánh giữa

Nút MAX đầu tiên:

$$V_1 = \max(-7, -2) = -2$$

Vì hai nhánh chưa xét có tổng xác suất $0.1 + 0.4 = 0.5$ và giá trị mỗi nhánh không vượt quá 20, giới hạn trên của Chance Node là:

$$U_M = 0.5(-2) + 0.5(20)$$

$$U_M = -1 + 10 = 9$$

Ta có:

$$9 < 17.5$$

Do đó, dù những nhánh chưa xét đạt giá trị tối đa, nhánh giữa vẫn không thể tốt hơn nhánh trái.

Cắt tỉa toàn bộ hai nhánh MAX chứa (-18, -20) và (0, 15).

Bước 3: Duyệt nhánh phải

MAX thứ nhất:

$$V_1 = \max(18, -9) = 18$$

MAX thứ hai có hai lá (20, -20). Sau khi xét lá đầu tiên là 20:

$$V_2 \geq 20$$

Mà giá trị MAX không thể vượt quá 20, suy ra:

$$V_2 = 20$$

Vì thế, cắt tỉa lá -20 trong MAX thứ hai.

MAX thứ ba:

$$V_3 = \max(-18, -10) = -10$$

Chance Node bên phải:

$$V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$$

Kết quả:

$$V_{root} = \max(17.5, 3.2, 7.6) = 17.5$$

Trong đó giá trị 3.2 của nhánh giữa là giá trị đầy đủ dùng để đối chiếu; khi thực sự cắt tỉa, thuật toán chỉ cần biết giới hạn trên bằng 9.

Kết quả câu c: Các nhánh có thể cắt tỉa là hai cây MAX cuối của nhánh giữa và lá -20 trong MAX thứ hai của nhánh phải. Giá trị tối ưu tại gốc vẫn là 17.5.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Forward Checking (FC) và AC-3 đều là các phương pháp lọc miền giá trị nhằm phát hiện sớm các xung đột ràng buộc trong bài toán CSP. Tuy nhiên, hai phương pháp khác nhau về phạm vi lan truyền, thời điểm áp dụng và chi phí tính toán.

Tiêu chí	Forward Checking	AC-3
Phạm vi	Chỉ lọc miền các biến chưa gán có ràng buộc trực tiếp với biến vừa gán	Lan truyền trên các cung của đồ thị ràng buộc đến khi đạt tính nhất quán cung
Thời điểm	Thường thực hiện sau mỗi phép gán	Có thể thực hiện trước khi tìm kiếm hoặc sau phép gán
Phát hiện xung đột	Chủ yếu phát hiện xung đột trực tiếp	Có thể phát hiện xung đột gián tiếp qua nhiều biến

Chi phí	Thường thấp hơn	Thường cao hơn do phải kiểm tra và cập nhật nhiều cung

Đối với CSP nhị phân có e cạnh ràng buộc và kích thước miền tối đa d, AC-3 có độ phức tạp thời gian trường hợp xấu nhất thường là:

$$O(ed^3)$$

Kết luận: Forward Checking có chi phí thấp nhưng phạm vi kiểm tra hạn chế, trong khi AC-3 lan truyền ràng buộc sâu hơn, giúp loại bỏ nhiều giá trị không hợp lệ và phát hiện xung đột sớm hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV là heuristic chọn biến có số lượng giá trị hợp lệ còn lại trong miền nhỏ nhất để thực hiện phép gán trước.

$$X^* = \arg\min_{X \in U} |D(X)|$$

Trong đó U là tập biến chưa được gán và $D(X)$ là miền hiện tại của biến X.

MRV tuân theo nguyên tắc Fail-First: ưu tiên xử lý biến khó nhất để phát hiện sớm những nhánh không có nghiệm, từ đó giảm số lần backtracking và số trạng thái cần duyệt.

ii) Least Constraining Value (LCV)

LCV là heuristic chọn giá trị gây ảnh hưởng ít nhất đến miền của các biến chưa được gán.

$$v^* = \arg\min_{v \in D(X)} N_{removed}(v)$$

Trong đó $N_{removed}(v)$ là số giá trị bị loại khỏi miền của các biến liên quan khi gán $X = v$.

LCV ưu tiên giá trị giữ lại nhiều lựa chọn nhất cho các biến còn lại, qua đó tăng khả năng tìm được nghiệm mà không cần backtracking.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến (Variables)

$$X = \{A, B, C, D, E\}$$

Mỗi biến đại diện cho một máy bay.

Miền giá trị (Domains)

Mỗi biến có giá trị là cặp gồm loại đường băng và khung giờ:

$$D(X_i) = R \times T$$

Trong đó:

$$R = \{Quốc\ Tế, Nội\ Địa\}$$
$$T = \{1, 2, 3, 4\}$$

Đặt $X_i = (r_i, t_i)$, với r_i là đường băng và t_i là khung giờ.

Ràng buộc đơn (Unary Constraints)

Máy bay B hạ cánh tại khung giờ 1:

$$t_B = 1$$

Máy bay D hạ cánh từ khung giờ 3 trở đi:

$$t_D \geq 3$$

Máy bay A chỉ có thể chờ đến khung giờ 2:

$$t_A \leq 2$$

Ràng buộc nhị phân (Binary Constraints)

Máy bay D phải hạ cánh trước khi máy bay C cất cánh:

$$t_D < t_C$$

Không có hai máy bay nào sử dụng cùng một đường băng trong cùng khung giờ:

$$\forall i \neq j, (r_i \neq r_j) \vee (t_i \neq t_j)$$

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Với ràng buộc bổ sung, A, B, C chỉ sử dụng đường băng quốc tế; D và E chỉ sử dụng đường băng nội địa.

Do đó, miền giá trị mới chỉ cần biểu diễn khung giờ:

$$D(A) = D(B) = D(C) = D(D) = D(E)$$
$$= \{1, 2, 3, 4\}$$

Các cạnh ràng buộc:

$$E = \{(A, B), (A, C), (B, C), (C, D), (D, E)\}$$

Trong đó, A–B, A–C, B–C và D–E là các ràng buộc khác khung giờ; C–D là ràng buộc thứ tự $D < C$.

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Bước 1: Áp dụng Unary Constraints

$$D(A) = \{1, 2\}$$
$$D(B) = \{1\}$$
$$D(C) = \{1, 2, 3, 4\}$$
$$D(D) = \{3, 4\}$$
$$D(E) = \{1, 2, 3, 4\}$$

Bước 2: Áp dụng AC-3

Vì B = 1, nên A không thể bằng 1 và C không thể bằng 1:

$$D(A) = \{2\}$$

$$D(C) = \{2, 3, 4\}$$

Vì A = 2, C không thể bằng 2:

$$D(C) = \{3, 4\}$$

Do $D \geq 3$ và $D < C$, ta loại C = 3 và D = 4:

$$D(C) = \{4\}, D(D) = \{3\}$$

Vì D và E cùng sử dụng đường băng nội địa, E không thể nhận giá trị 3:

$$D(E) = \{1, 2, 4\}$$

Kết quả câu c:

Biến	**1**	**2**	**3**	**4**	**Miền còn lại**
A	1	2	3	4	2
B	1	2	3	4	1
C	1	2	3	4	4
D	1	2	3	4	3
E	1	2	3	4	124

Kết quả trên cũng khớp với bài giải AC-3 của CMU.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Áp dụng Unary Constraints trước, sau đó thực hiện Forward Checking kết hợp MRV. Nếu có nhiều biến cùng số lượng giá trị còn lại, ưu tiên biến có ký tự nhỏ hơn và giá trị nhỏ hơn.

Bước	Phép gán	Diễn giải
1	(B, 1)	B có miền nhỏ nhất, chọn B = 1
2	(A, 2)	Sau khi loại A = 1, chỉ còn A = 2
3	(C, 3)	C và D cùng có 2 giá trị; ưu tiên C và thử giá trị 3
4	(C, 4)	C = 3 khiến miền D rỗng do $D < C$; hoàn tác và thử C = 4
5	(D, 3)	D phải nhỏ hơn C = 4 và lớn hơn hoặc bằng 3
6	(E, 1)	E không được trùng khung giờ với D, chọn giá trị nhỏ nhất là 1

Thứ tự các phép gán phát sinh:

$$(B, 1) \ -> \ (A, 2) \ -> (C, 3) \ -> (C, 4) \ -> (D, 3) \ -> \ (E, 1)$$

Phép gán $(C, 3)$ thất bại và phải hoàn tác một lần.

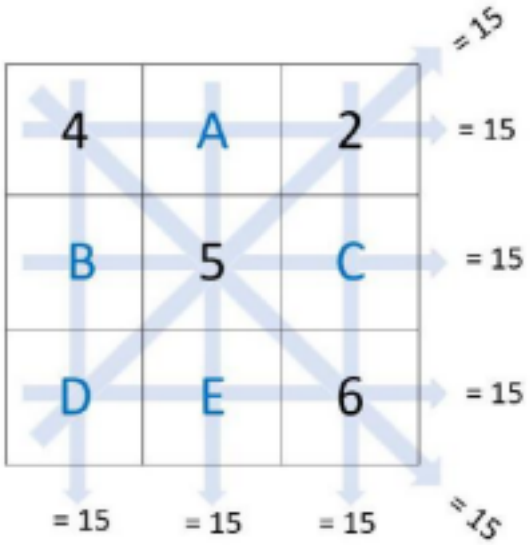
Nghiệm cuối cùng:

$$(A, B, C, D, E) \ = \ (2, 1, 4, 3, 1)$$

Nghiệm hợp lệ vì A, B, C có các khung giờ quốc tế khác nhau; D, E có các khung giờ nội địa khác nhau và D hạ cánh trước C cất cánh.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Theo hình Magic Square trong bài tập gốc, bảng 3 × 3 có dạng:

$$\begin{bmatrix} 4 & A & 2 \\ B & 5 & C \\ D & E & 6 \end{bmatrix}$$

Biến:

$$X = \{A, B, C, D, E\}$$

Miền giá trị:

Các giá trị từ 1 đến 9 không được trùng nhau. Vì 2, 4, 5, 6 đã được sử dụng:

$$D(A) = D(B) = D(C) = D(D) = D(E)$$

$$= \{1, 3, 7, 8, 9\}$$

Ràng buộc theo hàng:

$$4 + A + 2 = 15 \Rightarrow A = 9$$

$$B + 5 + C = 15 \Rightarrow B + C = 10$$

$$D + E + 6 = 15 \Rightarrow D + E = 9$$

Ràng buộc theo cột:

$$4 + B + D = 15 \Rightarrow B + D = 11$$

$$A + 5 + E = 15 \Rightarrow A + E = 10$$

$$2 + C + 6 = 15 \Rightarrow C = 7$$

Ràng buộc đường chéo:

$$4 + 5 + 6 = 15$$

$$2 + 5 + D = 15 \Rightarrow D = 8$$

Ràng buộc khác nhau: $AllDiff\ (A, B, C, D, E)$

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Bỏ qua ràng buộc AllDiff, các ràng buộc nhị phân còn lại tạo thành đồ thị:



Tập cạnh:

$$E = \{(A, E), (E, D), (D, B), (B, C)\}$$

Mỗi cạnh đại diện cho một phương trình tổng hai biến bằng một hằng số.

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Bước 1: Áp dụng các ràng buộc Unary

Từ tổng các hàng, cột và đường chéo:

$$A = 9, C = 7, D = 8$$

Miền sau khi lọc:

$$D(A) = \{9\}, D(C) = \{7\}$$

$$D(D) = \{8\}$$

$$D(B) = D(E) = \{1, 3, 7, 8, 9\}$$

Bước 2: Áp dụng AC-3

Xét ràng buộc giữa A và E:

$$A + E = 10$$

$$9 + E = 10 \Rightarrow E = 1$$

Xét ràng buộc giữa B và C:

$$B + C = 10$$

$$B + 7 = 10 \Rightarrow B = 3$$

Kiểm tra các ràng buộc còn lại:

$$D + E = 8 + 1 = 9$$

$$B + D = 3 + 8 = 11$$

Tất cả các ràng buộc đều thỏa mãn.

Miền cuối cùng:

Biến	**Miền sau AC-3**
A	9
B	3
C	7
D	8
E	1

Nghiệm Magic Square:



Mỗi hàng, cột và đường chéo đều có tổng bằng 15.

Nghiệm:

$$\begin{bmatrix} 4 & 9 & 2 \\ 3 & 5 & 7 \\ 8 & 1 & 6 \end{bmatrix}$$

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Bước 1: Chọn biến bằng MRV

Ban đầu:

$$D(AJ) = \{\text{Đỏ}, Vàng, Xanh\ Lá\}$$

$$D(Sunny) = \{\text{Đỏ}, Vàng, Hồng\}$$

$$D(Ellyse) = \{\text{Đỏ}, Hồng\}$$

Vì Ellyse có 2 giá trị, ít hơn AJ và Sunny (mỗi người 3 giá trị), ta chọn Ellyse.

Bước 2: Chọn giá trị bằng LCV

Nếu Ellyse chọn Đỏ, ta loại Đỏ khỏi AJ và Sunny, tổng số lựa chọn còn lại là $2 + 2 = 4$.

Nếu Ellyse chọn Hồng, chỉ loại Hồng khỏi Sunny, tổng số lựa chọn còn lại là $3 + 2 = 5$.

Vì $5 > 4$, chọn:

$$Ellyse = Hồng$$

Bước 3: Chọn biến tiếp theo

Sunny còn , trong khi AJ còn , nên chọn Sunny theo MRV.

Hai giá trị Đỏ và Vàng của Sunny đều loại đúng một giá trị khỏi miền AJ. Phá hòa theo thứ tự cầu vồng, chọn Đỏ:

$$Sunny = \text{Đỏ}$$

Bước 4: Gán cho AJ

AJ còn . Theo thứ tự cầu vồng, chọn Vàng:

$$AJ = Vàng$$

Kết quả:

$$Ellyse = Hồng$$
$$Sunny = \text{Đỏ}$$
$$AJ = Vàng$$

Thứ tự gán: Ellyse → Sunny → AJ.

Số lần backtracking: 0.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Bước 1: Chọn biến bằng Maximum Remaining Values

AJ và Sunny đều có 3 giá trị. Phá hòa theo thứ tự liệt kê trong đề, chọn AJ.

Bước 2: Chọn giá trị bằng Most Constraining Value

Nếu AJ chọn Đỏ, tổng số lựa chọn còn lại của Sunny và Ellyse là 3.

Nếu AJ chọn Vàng, tổng là 4.

Nếu AJ chọn Xanh lá, tổng là 5.

Chọn giá trị gây ràng buộc lớn nhất:

$$AJ = \text{Đỏ}$$

Bước 3: Chọn Sunny

Sau phép gán trên:

$$D(Sunny) = \{Vàng, Hồng\}$$
$$D(Ellyse) = \{Hồng\}$$

Sunny có 2 giá trị, nhiều hơn Ellyse có 1 giá trị, nên chọn Sunny.

Most Constraining Value ưu tiên Hồng vì loại giá trị duy nhất trong miền Ellyse:

$$Sunny = Hồng$$

Khi đó:

$$D(Ellyse) = \emptyset$$

Thuật toán gặp dead end.

Bước 4: Backtracking

Hoàn tác $Sunny = Hồng$, sau đó thử:

$$Sunny = Vàng$$

Ellyse nhận giá trị còn lại:

$$Ellyse = Hồng$$

Kết quả cuối cùng:

$$AJ = \text{Đỏ}$$
$$Sunny = Vàng$$
$$Ellyse = Hồng$$

Số lần backtracking: 1.

So sánh:

Heuristic	Số lần backtracking
MRV + LCV	0
Maximum Remaining Values + Most Constraining Value	1

Giải thích: MRV ưu tiên biến khó và LCV giữ lại nhiều lựa chọn cho các biến khác, giúp hạn chế xung đột. Ngược lại, Maximum Remaining Values và Most Constraining Value có thể làm cạn kiệt miền của biến chưa gán, dẫn đến dead end và phải backtracking. Kết quả này khớp với lời giải gốc của CMU, với quy ước ưu tiên AJ khi hòa số lượng giá trị.

--- Hết ---